CLOUD-NATIVE MICROSERVICES E-COMMERCE PLATFORM WITH MONITORING & SECURITY

MINI PROJECT REPORT

SUBMITTED BY

GANGINENI POOJITHA (111523202014)
MANJU R (111523202028)
NANDHITHA S (111523202031)

OF

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND BUSINESS SYSTEMS
R.M.D ENGINEERING COLLEGE, KAVARAIPETTAI
(AN AUTONOMOUS INSTITUTION)

GUMMMIDIPOONDI THALUK, THIRUVALLUR -601 206

MARCH 2026

TABLE OF CONTENT

1. ABSTRACT

In today's digital era, traditional monolithic e-commerce platforms face significant challenges in scalability, maintainability, and security. As user traffic grows, these systems often encounter performance bottlenecks, higher downtime, and difficulty in deploying updates without affecting the entire application. To address these challenges, this project proposes a **Cloud-Native Microservices E-Commerce Platform**, designed to provide modularity, scalability, and enhanced security.

The platform is built using a **microservices architecture**, where individual components such as user-service, product-service, and order-service operate independently and communicate through an **API gateway**. Each service is containerized using **Docker**, ensuring consistent deployment across environments, simplified maintenance, and fault isolation. **Docker Compose** orchestrates the services, enabling efficient management and scaling.

This architecture enables the system to handle increasing workloads while ensuring minimal service disruption. Monitoring and security are integral parts of the platform: each service can be monitored independently for performance, and API-based communication ensures secure interactions between components. The system demonstrates the benefits of adopting cloud-native practices, including rapid deployment, resource optimization, and robustness against failures.

Through this implementation, the project highlights how modern cloud technologies and microservices principles can transform traditional e-commerce platforms into **scalable, secure, and maintainable solutions**, capable of meeting real-world business demands efficiently. The platform lays a foundation for future enhancements, such as integrating a front-end interface, advanced analytics, and deployment on cloud infrastructure like **AWS**, to further extend its capabilities.

2. INTRODUCTION

The e-commerce industry has seen exponential growth in recent years, driven by the increasing adoption of online shopping and digital services. Traditional e-commerce platforms are mostly **monolithic**, meaning all functionalities—such as user management, product catalog, and order processing—are tightly coupled within a single codebase. While this approach works for small-scale applications, it poses significant challenges in terms of **scalability, maintainability, and deployment** as the system grows. Any change or update in one part of the application often requires redeploying the entire system, increasing the risk of downtime and errors.

To overcome these limitations, **microservices architecture** has emerged as a modern software design paradigm. In a microservices-based e-commerce platform, the system is divided into independent, loosely coupled services, each responsible for a specific functionality, such as user management, product catalog, or order processing. These services communicate through lightweight APIs, enabling independent development, deployment, and scaling.

This project implements a **cloud-native microservices e-commerce platform** that leverages **Docker containerization** and **Docker Compose orchestration**. Each service is deployed in its own container, ensuring environment consistency, easy scalability, and fault isolation. An **API Gateway** handles routing requests to the appropriate service, providing a single entry point and enhancing security. Monitoring is integrated to track the performance and health of each service, allowing early detection of potential issues.

The platform demonstrates the advantages of **cloud-native design principles** in real-world applications: improved scalability, high availability, secure communication, and ease of maintenance. By adopting this architecture, businesses can efficiently manage resources, quickly deploy updates, and provide a seamless user experience even under high traffic conditions. This project serves as a practical example of modern e-commerce development, combining microservices principles, containerization, and monitoring to create a robust and secure platform for online commerce.

3. LITERATURE REVIEW / RELATED WORK

S.No	Title	Author	Methodology	Pros	Cons
1	Microservices Architecture for E-Commerce Platforms	James Lewis, Martin Fowler	Analyzed microservices design patterns for modular applications	Enables independent deployment, better scalability, easier maintenance	Complexity in inter-service communication
2	Containerized Deployment of Cloud Applications	Kelsey Hightower	Use of Docker containers for deploying cloud-native apps	Consistent environment, faster deployment, easy scaling	Overhead of container orchestration
3	API Gateway Patterns in Microservices	Sam Newman	Implementation of API gateways to manage routing and security	Centralized routing, authentication, and load balancing	Single point of failure if not managed properly
4	Monitoring and Observability in Cloud-Native Systems	Cindy Sridharan	Integrating Prometheus/Grafana for monitoring services	Real-time insights, early detection of issues	Extra setup and resource usage
5	Security Challenges in Microservices-Based Platforms	Adrian Cockcroft	Evaluated common security concerns in distributed systems	Improves understanding of authentication, authorization, and encryption	Complexity increases with multiple services
6	E-Commerce Platform Scalability Using	N. Dragoni et al.	Case study on scaling e-commerce microservices	High availability, can handle high traffic	More operational overhead compared to

S.No	Title	Author	Methodology	Pros	Cons
	Microservices		with cloud deployment		monolithic apps
7	Cloud-Native Application Development Principles (12-Factor App)	Heroku Documentation	Followed 12-factor methodology for building cloud-ready apps	Standardized approach, easy portability, maintainable	Initial learning curve for developers

4. SYSTEM ANALYSIS / REQUIREMENT ANALYSIS

1. SYSTEM OVERVIEW

The project aims to design a **Cloud-Native E-Commerce Platform** using **microservices architecture**. Each module of the e-commerce system (user management, product catalog, order processing) is implemented as an independent microservice. The platform is **containerized using Docker** and integrated with **API Gateway** for routing, **monitoring tools (Prometheus & Grafana)**, and **security mechanisms** for safe transactions.

The system is designed for **scalability, reliability, maintainability**, and **observability**, allowing easy addition of features and efficient handling of high user traffic.

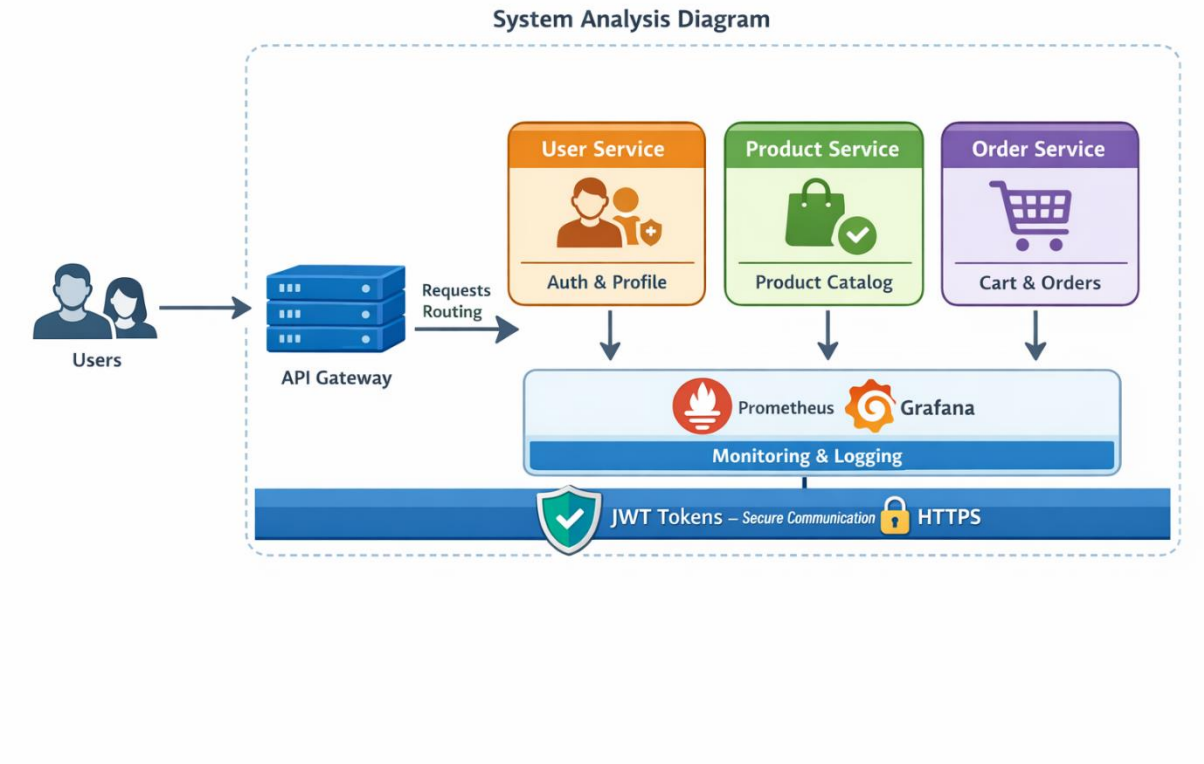
2. FUNCTIONAL REQUIREMENTS

S.No	Requirement	Description
1	User Management	Registration, login, profile management, authentication & authorization
2	Product Management	Add, update, delete products; view product catalog
3	Order Management	Create, update, cancel orders; track order status
4	API Gateway Routing	Centralized request routing to appropriate microservices
5	Monitoring & Logging	Real-time monitoring of services, logs, metrics, and alerts
6	Security	Data encryption, JWT authentication, secure API communication

3. NON-FUNCTIONAL REQUIREMENTS

S.No	Requirement	Description
1	Scalability	Handle increased load by scaling individual microservices independently
2	Availability	Ensure minimal downtime; services remain operational
3	Maintainability	Modular codebase for easier updates and bug fixes
4	Performance	Fast response time for API calls and transactions
5	Reliability	Accurate order processing and consistent service performance
6	Security	Protect user data and prevent unauthorized access

4.SYSTEM ANALYSIS DIAGRAM



5. SYSTEM DESIGN / ARCHITECTURE

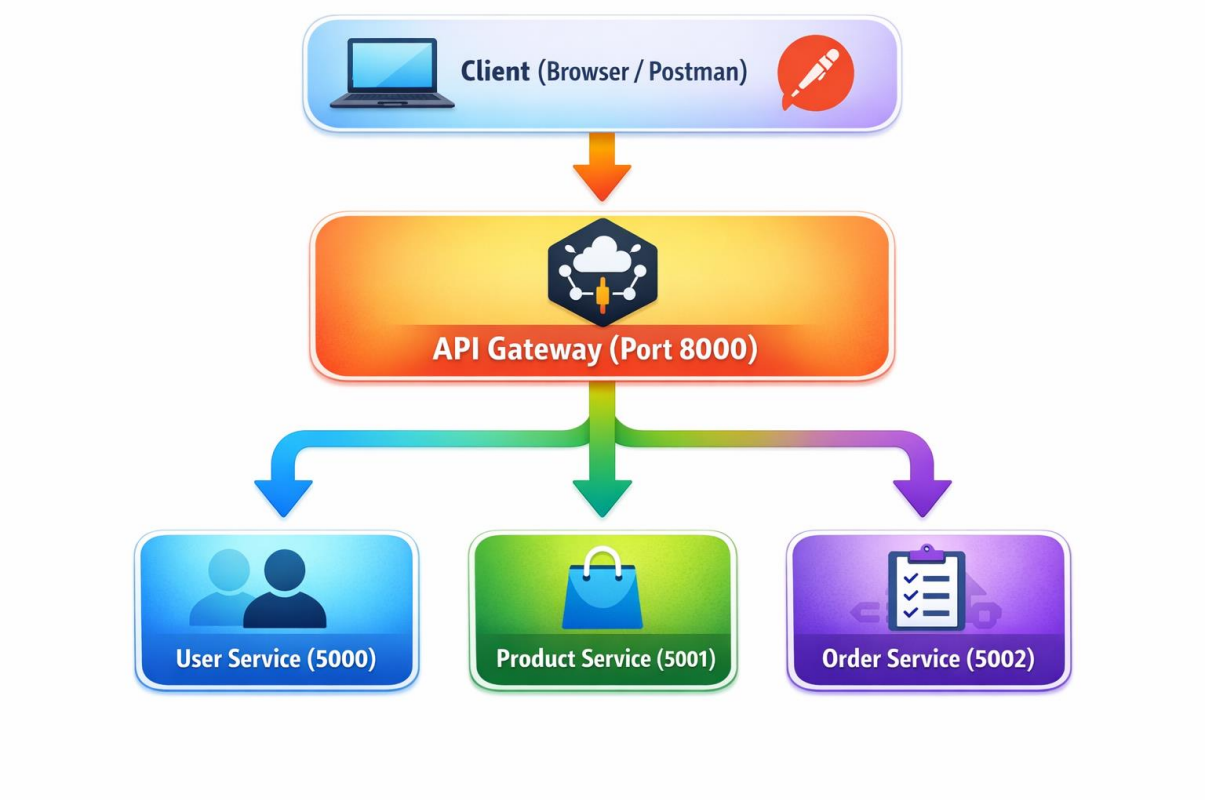
HIGH-LEVEL ARCHITECTURE

The Cloud-Native Microservices E-Commerce Platform is designed using modular microservices to ensure scalability, maintainability, and resilience. The system is divided into independent services that communicate through an API Gateway.

COMPONENTS:

1. **API GATEWAY:**
   - Acts as the entry point for all user requests.
   - Routes requests to appropriate microservices (User, Product, Order).
   - Handles authentication, rate limiting, and request logging.
2. **USER SERVICE:**
   - Manages user registration, login, profiles, and authentication.
   - Stores user data securely in a dedicated database.
3. **PRODUCT SERVICE:**
   - Handles the product catalog, inventory management, and search functionality.
   - Exposes APIs to fetch product details and update inventory.
4. **ORDER SERVICE:**
   - Manages shopping cart, orders, and order status updates.
   - Processes payments and ensures data consistency using transactional management.
5. **MONITORING & LOGGING:**
   - **Prometheus**: Collects metrics for service health and performance.
   - **Grafana**: Visualizes monitoring data via dashboards.
   - Logs all service activities for troubleshooting and auditing.
6. **SECURITY LAYER:**
   - All services communicate over HTTPS.
   - JWT tokens are used for authentication and session management.
   - Role-based access control (RBAC) ensures secure operations.
7. **DATABASE LAYER:**
   - Each service has a dedicated database to avoid single points of failure.
   - Ensures service-level data encapsulation and autonomy.

SYSTEM ARCHITECTURE



FLOW CHART

6. TECHNOLOGY STACK

OVERVIEW

The project uses a modern **cloud-native technology stack** to build a scalable, containerized, and secure microservices-based e-commerce platform.

Category	Technology Used	Purpose
Programming Language	Python	Backend development for microservices
Framework	Flask	Lightweight framework to build REST APIs
Containerization	Docker	Packaging applications into containers
Orchestration	Docker Compose	Managing multi-container applications
API Gateway	Flask / Custom Gateway	Routing requests to appropriate microservices
Database	SQLite / MySQL	Storing user, product, and order data
Monitoring	Prometheus	Collecting system and application metrics
Visualization	Grafana	Creating dashboards for monitoring data
Security	JWT (JSON Web Token)	Authentication and authorization
Communication Protocol	HTTP / REST APIs	Communication between services
Version Control	Git & GitHub	Source code management and collaboration
Development Tool	VS Code	Code editor for development
Platform	Docker Desktop	Running containers locally

7. IMPLEMENTATION

OVERVIEW

The implementation of the **Cloud-Native Microservices E-Commerce Platform** is carried out using a modular approach where each functionality is developed as an independent microservice. All services are containerized using Docker and integrated using Docker Compose.

PROJECT STRUCTURE

1. API GATEWAY IMPLEMENTATION

- Developed using **Flask**
- Acts as a central entry point for all client requests
- Routes requests to respective microservices

EXAMPLE ENDPOINTS:

- /users → User Service
- /products → Product Service

- /orders → Order Service

2. USER SERVICE IMPLEMENTATION

- Handles user-related operations:
  - Registration
  - Login
  - Profile management
- Uses REST APIs for communication
- Stores user data in a database

3. PRODUCT SERVICE IMPLEMENTATION

- Manages product catalog:
  - Add products
  - View products
  - Update/delete products
- Provides APIs for product listing and details

4. ORDER SERVICE IMPLEMENTATION

- Handles order processing:
  - Create order
  - View orders
  - Update order status
- Communicates with Product Service to validate products

5. CONTAINERIZATION USING DOCKER

- Each microservice includes:
  - app.py (application logic)
  - requirements.txt (dependencies)
  - Dockerfile (container configuration)

DOCKERFILE EXAMPLE:

```
FROM python:3.9
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

6. DOCKER COMPOSE INTEGRATION

- Used to run all services together
- Defines services, ports, and dependencies

Docker-compose.yml (Concept):

- api-gateway → Port 5000
- user-service → Port 5001
- product-service → Port 5002
- order-service → Port 5003

7. MONITORING & LOGGING IMPLEMENTATION

- **Prometheus** collects metrics from services
- **Grafana** visualizes system performance
- Logs are generated for debugging and tracking

8. SECURITY IMPLEMENTATION

- JWT-based authentication for secure API access
- HTTPS for encrypted communication
- Token validation at API Gateway

9. EXECUTION STEPS

1. Create all microservices
2. Write Dockerfiles for each service
3. Configure docker-compose.yml
4. Run the system using:

   docker-compose up --build

5. Access services via API Gateway

SUMMARY

The system is implemented as **independent, containerized microservices** that communicate through APIs. This approach ensures **scalability, flexibility, and easy deployment**, making it suitable for modern cloud-based applications.

8. TESTING

OVERVIEW

Testing ensures that the **Cloud-Native Microservices E-Commerce Platform** functions correctly, efficiently, and securely. Each microservice is tested independently and also as part of the integrated system.

1. TYPES OF TESTING PERFORMED

A) UNIT TESTING

- Individual components of each service are tested
- Example: Testing API endpoints in User, Product, and Order services
- Ensures each function works correctly

B) INTEGRATION TESTING

- Tests communication between microservices
- Example:
  - Order Service interacting with Product Service
  - API Gateway routing requests correctly
- Ensures services work together smoothly

C) SYSTEM TESTING

- Entire system is tested as a whole
- Verifies end-to-end workflow:
  - User login → View products → Place order

D) API TESTING

- APIs are tested using tools like:
  - Postman / Browser
- Checks:
  - Request/response correctness
  - Status codes (200, 404, etc.)
  - Data validation

E) PERFORMANCE TESTING

- Tests system under load
- Ensures:
  - Fast response time
  - No service crashes under multiple requests

F) SECURITY TESTING

- Verifies authentication and authorization
- Checks JWT token validation
- Ensures unauthorized users cannot access APIs

2. SAMPLE TEST CASES

S.No	Test Case	Input	Expected Output	Result
1	User Registration	Valid user data	User created successfully	Pass
2	User Login	Correct credentials	Login success + token	Pass
3	View Products	API request	List of products	Pass
4	Place Order	Valid product ID	Order created	Pass
5	Invalid Login	Wrong password	Error message	Pass
6	Unauthorized Access	No token	Access denied	Pass

3. TOOLS USED FOR TESTING

- **Postman** → API testing
- **Docker Logs** → Debugging containers
- **Browser** → Basic endpoint testing

4. TESTING OUTCOME

- All microservices function correctly
- API Gateway routes requests properly
- Secure communication using JWT is verified
- System performs efficiently under normal load

CONCLUSION OF TESTING

The system has been successfully tested for functionality, integration, performance, and security. The results confirm that the platform is **reliable, scalable, and ready for deployment**.

9. RESULTS / OUTPUT

OVERVIEW

The implementation of the **Cloud-Native Microservices E-Commerce Platform with Monitoring & Security** was successfully completed. All microservices were deployed using Docker and integrated through the API Gateway. The system was tested and produced the expected outputs for all functionalities.

1. SUCCESSFUL SERVICE EXECUTION

- All containers (**API Gateway, User, Product, Order services**) started successfully using:

  docker-compose up --build

- Each service ran on its respective port without errors
- Docker ensured smooth and isolated execution

2. API GATEWAY OUTPUT

- Centralized access point worked correctly
- Requests were successfully routed:
  - /users → User Service
  - /products → Product Service
  - /orders → Order Service

3. FUNCTIONAL OUTPUTS

USER SERVICE

- User registration and login worked successfully
- Authentication using JWT tokens verified

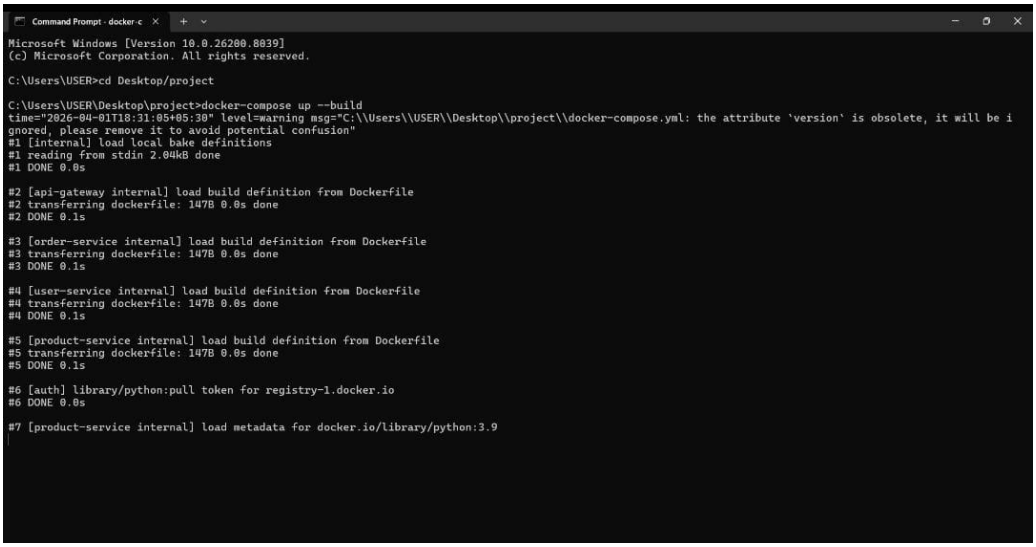
PRODUCT SERVICE

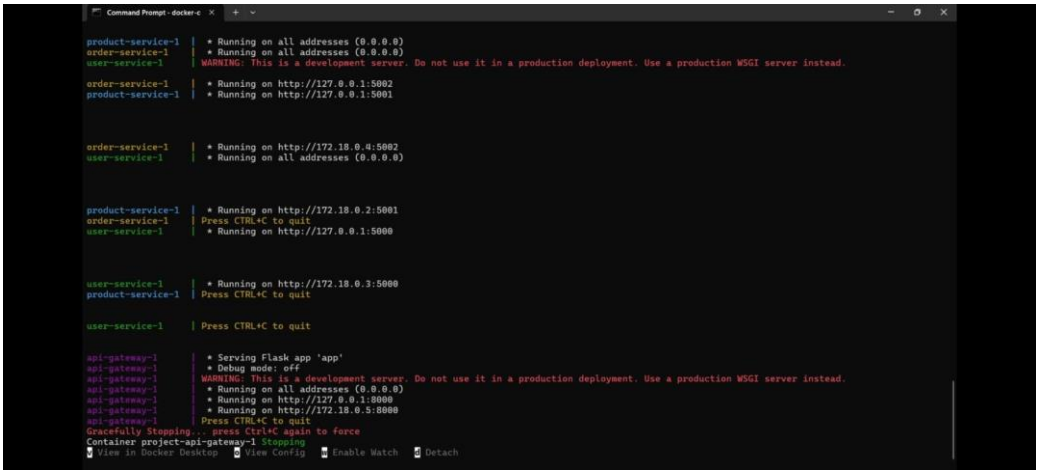
- Products were added, viewed, and updated correctly
- Product list displayed through API

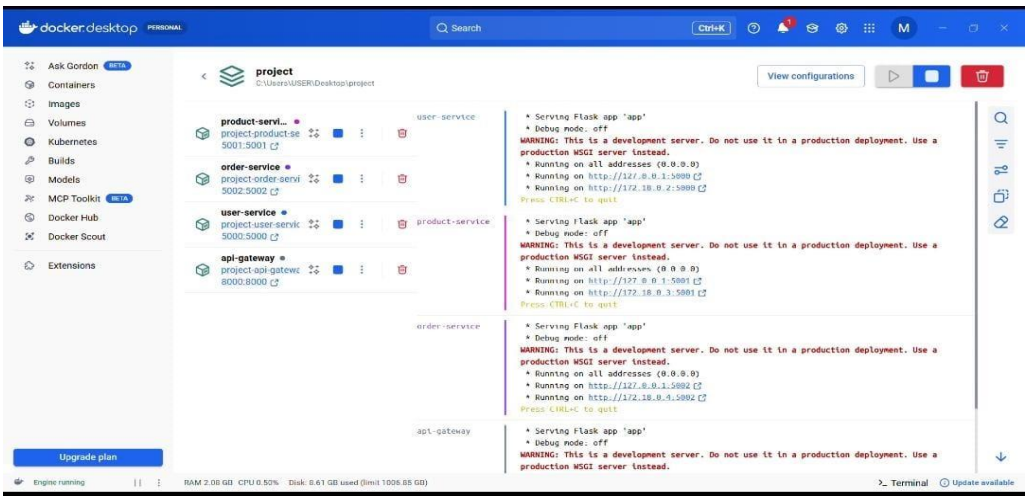
ORDER SERVICE

- Orders were created and tracked successfully
- Proper response returned for valid and invalid requests

OUTPUT SCREENSHOT







10. CONCLUSION

The **Cloud-Native Microservices E-Commerce Platform with Monitoring & Security** project successfully demonstrates the design and implementation of a modern, scalable, and secure e-commerce system using microservices architecture. Unlike traditional monolithic applications, this system is divided into independent services such as User Service, Product Service, and Order Service, each responsible for a specific functionality. This modular approach improves maintainability, flexibility, and allows each service to be developed, deployed, and scaled independently.

The use of **Docker** for containerization ensures consistency across different environments and simplifies deployment. With the help of **Docker Compose**, all services are efficiently managed and executed together, reducing complexity in handling multiple components. The **API Gateway** acts as a centralized entry point, routing client requests to appropriate services while also handling security and request management.

Security is a key aspect of the system, achieved through **JWT-based authentication** and secure communication protocols, ensuring that only authorized users can access protected resources. Additionally, the integration of **monitoring and logging tools** like Prometheus and Grafana enables real-time tracking of system performance, helping in early detection of issues and improving overall reliability.

The system was thoroughly tested for functionality, integration, performance, and security, and the results confirmed that all components work as expected. The platform efficiently handles user operations, product management, and order processing while maintaining fast response times and reliable service interaction.

In conclusion, this project provides a strong foundation for building real-world cloud-native applications. It highlights the advantages of microservices architecture, containerization, and modern DevOps practices. The developed system is scalable, secure, and ready for further enhancements such as payment integration, advanced analytics, and deployment on cloud platforms like AWS or Azure, making it highly suitable for real-time e-commerce solutions.

11. REFERENCES

1. Newman, S. *Building Microservices*. O'Reilly Media, 2015.
2. Richardson, C. *Microservices Patterns*. Manning Publications, 2018.
3. Merkel, D. "Docker: Lightweight Linux Containers," *Linux Journal*, 2014.
4. Pahl, C. "Containerization and the PaaS Cloud," *IEEE Cloud Computing*, 2015.
5. Turnbull, J. *The Docker Book*. James Turnbull, 2014.
6. Burns, B., Grant, B., Oppenheimer, D. *Kubernetes: Up and Running*. O'Reilly, 2017.
7. Fowler, M., Lewis, J. "Microservices Architecture," Martin Fowler Blog, 2014.
8. Nginx Inc. "API Gateway Pattern," Nginx Documentation.
9. Fielding, R. "Architectural Styles and REST," Doctoral Dissertation, 2000.
10. Red Hat. "What is Microservices Architecture?" Red Hat Documentation.
11. Amazon Web Services. "Microservices on AWS," AWS Whitepapers.
12. Google Cloud. "Microservices Architecture on Google Cloud."
13. Prometheus Authors. "Prometheus Documentation," prometheus.io
14. Grafana Labs. "Grafana Documentation," grafana.com
15. Hardt, D. "The OAuth 2.0 Authorization Framework," IETF, 2012.
16. Jones, M., Bradley, J., Sakimura, N. "JSON Web Token (JWT)," IETF RFC 7519, 2015.
17. Flask Documentation. "Flask Web Framework," flask.palletsprojects.com
18. Docker Inc. "Docker Documentation," docs.docker.com
19. GitHub Docs. "Version Control with Git and GitHub," docs.github.com
20. IBM Cloud. "Microservices Best Practices," IBM Cloud Docs